CS220 Lab#7
32-bit MIPS Processor
(Part-1: Arithmetic and logic instructions)

Mainak Chaudhuri
Indian Institute of Technology Kanpur

Sketch

- Milestone#1: Single-cycle implementation of a MIPS processor
  - Carries three marks
- Milestone#2: Two-cycle implementation of a MIPS processor
  - Carries two marks
- Milestone#3: Three-cycle implementation of a MIPS processor
  - Carries two marks
- At each milestone show all your work to a TA to receive marks

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The MIPS processor that you will design should support the add, sub, addi, sll, srl, sra, sllv, srlv, srav, and, or, xor, nor, andi, ori, xori, syscall instructions (syscall instruction discussed later)
  - Each MIPS instruction is 32 bits long
  - The instructions that have only register operands (add, sub, and, or, xor, nor, sll, sllv, srl, srlv, sra, srav, syscall) have the following format

    - opcode is 6'b0 for all instructions

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The instructions that have only register operands (add, sub, and, or, xor, nor, sll, sllv, srl, srlv, sra, srav, syscall) have the following format

    - If we declare the register file of the processor as the array reg [31:0] regfile [0:31], the operation is defined as regfile[rd] <= regfile[rs] *op* regfile[rt] or regfile[rd] <= regfile[rs] *op* sh_amt
      - Operation *op* is encoded using the function field as given in the parentheses: sll (6'h0), srl (6'h2), sra (6'h3), sllv (6'h4), srlv (6'h6), srav (6'h7), syscall (6'hc), add (6'h20), sub (6'h22), and (6'h24), or (6'h25), xor (6'h26), nor (6'h27)

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The instructions that have only register operands (add, sub, and, or, xor, nor, sll, sllv, srl, srlv, sra, srav, syscall) have the following format

    - The sh_amt field is used only in sll, srl, and sra instructions
      - This is the shift amount e.g., shift amount would be 20 in the operation (x << 20)

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The instructions that have only register operands (add, sub, and, or, xor, nor, sll, sllv, srl, srlv, sra, srav, syscall) have the following format

    - The syscall instruction is used to communicate with the operating system
      - We will use it to obtain program output (a proxy for printing) and to exit/terminate a program
      - We will use regfile[rs] to specify the syscall operation number (1001 for exit and 1004 for output)
      - When the syscall operation number is 1004, the value of regfile[rt] is copied into a circular I/O register array

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The instructions that have an immediate operand (addi, andi, ori, xori) have the following format

- The opcodes are as follows: addi (6'h8), andi (6'hc), ori (6'hd), xori (6'he)
- The immediate field is sign-extended for arithmetic operations (e.g., addi) and zero-extended for logical operations (e.g., andi, ori, xori) to create a 32-bit operand
- The operation is defined as regfile[rt] <= regfile[rs] *op* (zero- or sign-extended immediate)

Milestone#1

- Single-cycle implementation of a MIPS processor
  - Let us compile a simple example program

```
int a=-15, b=20;
c=a+b;
print c;
```

  - Translation to MIPS assembly language

```
addi $1, $0, -15    // a=-15
addi $2, $0, 20     // b=20
add $3, $1, $2      // c=a+b
addi $4, $0, 1004   // $4 has print syscall number
syscall $4, $3      // print c
addi $4, $0, 1001   // $4 has exit syscall number
syscall $4, $0      // exit code 0 (please ignore)
```

Milestone#1

- Single-cycle implementation of a MIPS processor
  - Let us compile a simple example program

```
int a=-15, b=20;
c=a+b;
print c;
```

  - Translation to MIPS assembly and machine language

```
addi $1, $0, -15    // Op: 001000, rs=0, rt=1, imm=0xfff1
addi $2, $0, 20     // Op: 001000, rs=0, rt=2, imm=0x0014
add $3, $1, $2      // Op: 0, rs=1, rt=2, rd=3, func=100000
addi $4, $0, 1004   // Op: 001000, rs=0, rt=4, imm=0x03ec
syscall $4, $3      // Op: 0, rs=4, rt=3, func=001100
addi $4, $0, 1001   // Op: 001000, rs=0, rt=4, imm=0x03e9
syscall $4, $0      // Op: 0, rs=4, func=001100
```

Milestone#1

- Single-cycle implementation of a MIPS processor
  - Let us compile a simple example program

```
int a=-15, b=20;
c=a+b;
print c
```

  - Translation to MIPS instruction encoding

```
addi $1, $0, -15    // instruction=0x2001fff1
addi $2, $0, 20     // instruction=0x20020014
add $3, $1, $2      // instruction=0x00221820
addi $4, $0, 1004   // instruction=0x200403ec
syscall $4, $3      // instruction=0x0083000c
addi $4, $0, 1001   // instruction=0x200403e9
syscall $4, $0      // instruction=0x0080000c
```

Milestone#1

- Single-cycle implementation of a MIPS processor
  - Design of the processor includes an arithmetic logic unit (ALU) to implement the operations and a register file
  - In a cycle, the processor fetches an instruction on posedge of clock, decodes it to extract different fields, looks up register file to get the operands, executes the operation in the ALU, and writes the result back to the register file

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The register file module should have two read ports, a write port and a write enable signal
    - Read operations are combinational, but write is sequential
    - Since register write needs to complete within the same clock cycle as instruction fetch, it should be done on negedge of clock
  - The register file module is declared below
    - module RegisterFile (input [4:0] read_addr1, input [4:0] read_addr2, output [31:0] read_data1, output [31:0] read_data2, input [4:0] write_addr, input [31:0] write_data, input write_enable, input clk);

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The register file module is shown below partly

```
module RegisterFile (input [4:0] read_addr1, input [4:0]
read_addr2, output [31:0] read_data1, output [31:0]
read_data2, input [4:0] write_addr, input [31:0] write_data,
input write_enable, input clk);
   reg [31:0] regfile [0:31];
   assign read_data1 = …  // Used as src1 input of ALU
   assign read_data2 = …  // Used in src2 input of ALU
   always @ (negedge clk) begin
      if (write_enable && (write_addr != 0)) begin
         …  // write to regfile[write_addr]
      end
   end
endmodule
```

Milestone#1

- Single-cycle implementation of a MIPS processor
  - All constants should be put in a header file
  - To do this, when creating a source file, select *Verilog Header* as the file type
    - Name it defs.vh
    - Include the following in defs.vh

```
`define OP_REG 6'h0
`define OP_ADDI 6'h8
`define OP_ANDI 6'hc
`define OP_ORI 6'hd
`define OP_XORI 6'he
```

Milestone#1

- Single-cycle implementation of a MIPS processor
  - Include the following in defs.vh

```
`define FUNC_SLL 6'h0
`define FUNC_SRL 6'h2
`define FUNC_SRA 6'h3
`define FUNC_SLLV 6'h4
`define FUNC_SRLV 6'h6
`define FUNC_SRAV 6'h7
`define FUNC_SYSCALL 6'hc
`define FUNC_ADD 6'h20
`define FUNC_SUB 6'h22
`define FUNC_AND 6'h24
`define FUNC_OR 6'h25
`define FUNC_XOR 6'h26
`define FUNC_NOR 6'h27
`define READ_COMMAND 1'b0    // Memory read command
`define WRITE_COMMAND 1'b1   // Memory write command
`define SYS_exit 32'd1001    // Syscall number for exit
`define SYS_write 32'd1004   // Syscall number for print
```

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The ALU module is completely combinational and should be implemented as a combinational always block

```
`include "defs.vh"
module ALU (input [31:0] src1, input [31:0] src2, input [4:0]
shift_amount, input [5:0] opcode, input [5:0] func, output [31:0] dest,
output dest_valid);
   reg [31:0] result;
   reg result_valid;
   assign dest = result;  // Used as write data for register file (RF)
   assign dest_valid = result_valid;  // Used as write enable for RF
   always @(src1 or src2 or opcode or func or shift_amount) begin
      case (opcode)
         `OP_REG: begin
           case (func)
              `FUNC_SLL: begin
                 result = src1 << shift_amount;
                 result_valid = 1'b1;
              end
              `FUNC_SRL: begin
                 ....
```

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The ALU module is completely combinational and should be implemented as a combinational always block
    - Use +, –, >>, <<, &, |, ^, ~ operators of Verilog to implement the operations
    - Use the >>> operator to write the Verilog code for shift right arithmetic and type cast src1 to signed

      result = $signed(src1) >>> …

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The processor module instantiates a register file module and an ALU module
  - We also assume that the circular I/O register array has four entries meaning that four variables can be printed in a program
  - The processor module interfaces with an external memory module for fetching instructions
    - Sends program counter (PC) to the memory module and receives the corresponding instruction
  - The structure of the processor module is shown over the next few slides

Milestone#1

- Single-cycle implementation of a MIPS processor

```
`include "defs.vh"
module Processor(input clk, output halt, input reset, output reg [7:0]
pc, input [31:0] ins, output [31:0] io_reg1, output [31:0] io_reg2,
output [31:0] io_reg3, output [31:0] io_reg4);
   wire [5:0] opcode;          // Extracted from ins
   wire [5:0] func;            // Extracted from ins
   wire [4:0] shift_amount;    // Extracted from ins
   wire [4:0] src1_addr;       // rs extracted from ins (input to RF)
   wire [4:0] src2_addr;       // rt extracted from ins (input to RF)
   wire [31:0] src1;           // Output of RF, input to ALU
   wire [31:0] src2;           // Output of RF, input to ALU
   wire [4:0] dest_addr;       // rt/rd extracted from ins (input to RF)
   wire [31:0] dest_data;      // Output of ALU, input to RF
   wire dest_data_valid;       // Output of ALU, input to RF
   wire [7:0] next_pc;         // Next instruction address
   wire [15:0] imm;            // Immediate extracted from ins
   reg [31:0] io_reg [0:3];    // Circular I/O buffer
   reg [1:0] io_reg_index;     // Index to circular I/O buffer
   reg fetched;                // Is first instruction fetched?
```

Milestone#1

- Single-cycle implementation of a MIPS processor

```
assign io_reg1 = io_reg[0];
assign io_reg2 = io_reg[1];
assign io_reg3 = io_reg[2];
assign io_reg4 = io_reg[3];
RegisterFile rf (…, …, …, …, …, …, dest_data_valid & fetched, clk);
ALU alu (src1, …, shift_amount, opcode, func, dest_data, dest_data_valid);
assign next_pc = (fetched & ~halt) ? pc + 1 : 8'b0;
always @(posedge clk) begin
   if (reset) begin
      pc <= 8'b0;
      io_reg_index <= 2'b0;
      fetched <= 1'b0;
   end
   else begin
      pc <= halt ? pc : next_pc;
      fetched <= 1'b1;
   end
end
```

-

Milestone#1

- Single-cycle implementation of a MIPS processor

```
always @(negedge clk) begin
  if ((opcode == `OP_REG) && (func == `FUNC_SYSCALL) &&
(src1 == `SYS_write)) begin
    io_reg_index <= io_reg_index + 1;
    io_reg[io_reg_index] <= src2;
  end
end
// Decode instruction
assign opcode = ins [31:26];
assign src1_addr = …
assign src2_addr = …
assign dest_addr = …
assign shift_amount = …
assign func = …
assign imm = …
assign halt = (reset | ~fetched) ? 1'b0 : (((opcode == `OP_REG)
&& (func == `FUNC_SYSCALL) && (src1 == `SYS_exit)) ? 1'b1 :
1'b0);
endmodule  // end of Processor module
```

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The memory module is used to store instructions and data
    - For simplicity, the memory is word-addressable
      - Each address refers to a 32-bit chunk
      - Works perfectly for instructions because each instruction is four bytes
      - Needs extra work for doing sub-word data access
    - Instructions are laid out starting from address zero and after all instructions are stored, data words are laid out
    - In this lab, we will use the memory module for storing and fetching instructions only

Milestone#1

- Single-cycle implementation of a MIPS processor
  - The memory module takes a command (read or write) as input along with a word address
    - If the command is write, it also takes a 32-bit word as input
    - If the command is read, it produces a 32-bit word as output
  - The structure of the memory module is shown in the next slide
    - Reading is combinational, while writing is done on posedge clk
    - We will model a 256-word memory (enough for small programs)

Milestone#1

- Single-cycle implementation of a MIPS processor

```
`include "defs.vh"
module Memory(input write_enable, input clk, input
command, input [7:0] address, input [31:0] word_in, output
[31:0] word_out);
   reg [31:0] Mem [0:255];
   assign word_out = (command == `READ_COMMAND) ? …
: …;
   always @ (posedge clk) begin
      if ((command == `WRITE_COMMAND) &&
(write_enable == 1'b1)) begin
         …   // Store word_in
      end
   end
endmodule
```

Milestone#1

- Single-cycle implementation of a MIPS processor
  - Top-level module (call it Computer) instantiates a Processor module and a Memory module
  - It takes a sequence of instructions and their addresses as inputs (one <instruction, address> pair at a time) and stores the instructions in memory
  - Once all instructions are stored (indicated by a *done_storing* input from environment), it asks the processor to execute the stored instructions
  - Outputs a *done* signal, the total number of cycles, the number of cycles required by the processor to execute the instructions, and the I/O outputs produced by the processor

Milestone#1

- Single-cycle implementation of a MIPS processor
  - Structure of the Computer module

```
`include "defs.vh"
module Computer(input reset, input [7:0] ins_addr, input
[31:0] ins, input clk, input done_storing, output reg done,
output [31:0] out_reg1, output [31:0] out_reg2, output
[31:0] out_reg3, output [31:0] out_reg4, output [31:0]
total_cycles, output [31:0] proc_cycles);
   wire [7:0] pc;                  // Output of Processor
   wire [31:0] ins_fetched;        // Output of Memory
   wire ins_mem_command;           // Input to Memory
   reg [31:0] counter_total;       // Counts total_cycles
   reg [31:0] counter_proc;        // Counts proc_cycles
   wire halt;                      // Output of Processor
```

Milestone#1

- Single-cycle implementation of a MIPS processor

```
Memory mem(~reset & ~done_storing, clk,
ins_mem_command, done_storing ? pc : ins_addr, ins,
ins_fetched);

Processor proc(clk, halt, ~done_storing, pc,
ins_fetched, out_reg1, out_reg2, out_reg3, out_reg4);

assign total_cycles = counter_total;

assign proc_cycles = counter_proc;

assign ins_mem_command = done_storing ?
`READ_COMMAND : `WRITE_COMMAND;
```

Milestone#1

- Single-cycle implementation of a MIPS processor

```
always @(posedge clk) begin
  if (reset) begin
    counter_total <= 32'b0;
    counter_proc <= 32’b0;
    done <= 1'b0;
  end
  else begin
    done <= …
    counter_total <= …
    counter_proc <= …
  end
end
endmodule
```

Milestone#1

- Single-cycle implementation of a MIPS processor
  - Write a simulation testbench for the Computer module to run the small program that we wrote in slide 10 and verify that out_reg1 contains 5 at the end; verify the output cycle counts also
    - Generate a clock of period 10 and 50% duty
    - Initially, reset=1, done_storing=0
    - At time=7, make reset=0, set the first instruction and ins_addr=0
    - After 10 time units, set the second instruction and ins_addr=1
    - Continue until all seven instructions are input
    - After 10 time units, set done_storing=1

Milestone#1

- Single-cycle implementation of a MIPS processor
  - Create AXI4 IP, synthesize, implement
    - Implementation may report timing failure
    - It fails to complete all the work within a cycle
      - The processor gets just half cycle to fetch an instruction, decode it, look up register file to fetch operands, and execute the operation in ALU
      - The remaining half cycle is devoted to execute print system call and register file write
    - Examine the failing paths from timing report
      - Try to understand the longest paths (these are the critical paths of the processor)
    - Show simulation output to a TA and move on to the next milestone

Milestone#1

- Single-cycle implementation of a MIPS processor
  - Depending on your design, it may be possible that implementation completes successfully without any timing error
  - In that case, generate bit stream, export hardware, show that your design works by writing a C program in Vitis IDE
    - The relevant C code segment is shown in the next three slides
    - You can try changing the values of a and b
    - You can try writing a different program to run on the MIPS processor (e.g., replace c=a+b by c=a – b)

Milestone#1

- Vitis IDE C program segment

```
unsigned reset=1, ins_addr, ins, done_storing=0, done=0,
total_cycles, proc_cycles;
int c, a = -15, b = 20;
Xil_Out32(base_addr, reset);
Xil_Out32(base_addr+3, done_storing);
reset=0;
Xil_Out32(base_addr, reset);
ins_addr = 0;
ins = (0x2001 << 16) | (a & 0xffff);
Xil_Out32(base_addr+1, ins_addr);
Xil_Out32(base_addr+2, ins);
ins_addr++;
ins = (0x2002 << 16) | (b & 0xffff);
Xil_Out32(base_addr+1, ins_addr);
Xil_Out32(base_addr+2, ins);
ins_addr++;
ins = 0x00221820;
Xil_Out32(base_addr+1, ins_addr);
Xil_Out32(base_addr+2, ins);
```

Milestone#1

- Vitis IDE C program segment

```
ins_addr++;
ins = 0x200403ec;
Xil_Out32(base_addr+1, ins_addr);
Xil_Out32(base_addr+2, ins);
ins_addr++;
ins = 0x0083000c;
Xil_Out32(base_addr+1, ins_addr);
Xil_Out32(base_addr+2, ins);
ins_addr++;
ins = 0x200403e9;
Xil_Out32(base_addr+1, ins_addr);
Xil_Out32(base_addr+2, ins);
ins_addr++;
ins = 0x0080000c;
Xil_Out32(base_addr+1, ins_addr);
Xil_Out32(base_addr+2, ins);
```

Milestone#1

- Vitis IDE C program segment

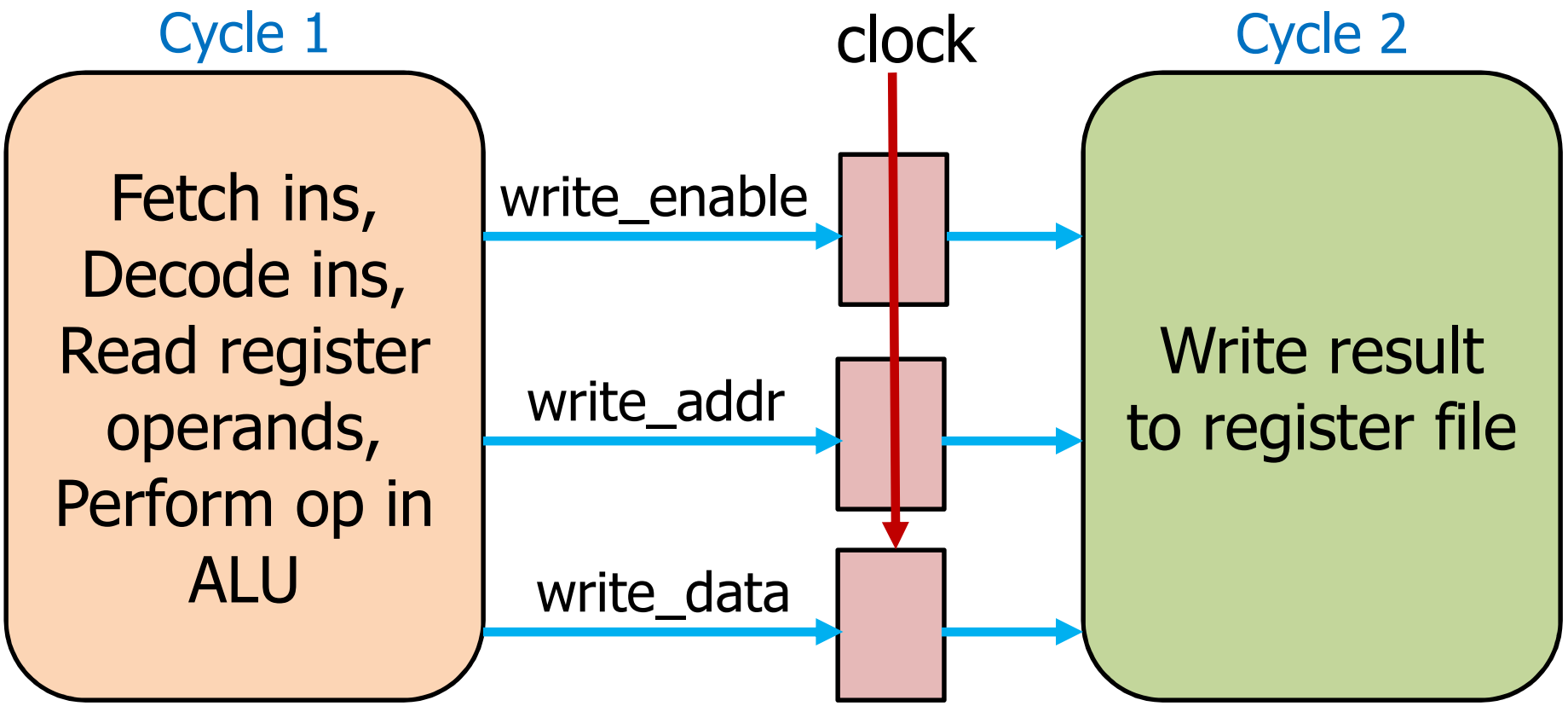
```
done_storing = 1;
Xil_Out32(base_addr+3, done_storing);
while (!done) done = Xil_In32(base_addr+4);
c = Xil_In32(base_addr+5);
total_cycles = Xil_In32(base_addr+9);
proc_cycles = Xil_In32(base_addr+10);
xil_printf("\n\r(%d)+(%d)=%d, total cycles: %u,
computation cycles: %u\n\r", a, b, c, total_cycles,
proc_cycles);
printf("Processor cycles per instruction (CPI): %f\n\r",
((float)proc_cycles)/(ins_addr+1));
```

Milestone#2

- Two-cycle implementation of a MIPS processor
  - A natural circuit cut is before register write
    - Register write can be moved to the next cycle
    - Allows the processor to use one full cycle to fetch an instruction, decode it, read register file, and execute the operation in ALU
    - This design requires two cycles to complete an instruction
    - A new instruction can be fetched only after the previous instruction is completed
  - Make minimal changes to the modules to implement this idea

Milestone#2

- Two-cycle implementation of a MIPS processor
  - Functional decomposition across cycles in the two-cycle design



Milestone#2

- Two-cycle implementation of a MIPS processor
  - You may think of the processor as a finite state machine (FSM) with two states
    - State 0: fetch, decode, read from RF, execute
    - State 1: write to RF
    - In these FSMs, the inputs to a state may include the outputs of the previous states (e.g., write_enable, write_addr, write_data produced in state 0 are used in state 1)
    - The signals write_enable, write_addr, and write_data must be copied into inter-state registers to cut the critical path and carry them from one cycle to the next
    - Multi-cycle designs need these inter-state registers

Milestone#2

- Two-cycle implementation of a MIPS processor
  - You should not need to change the Computer module's inputs and outputs
  - Verify functional correctness of the two-cycle design using the same simulation testbench developed in the previous milestone
  - Show the simulation output to a TA

Milestone#2

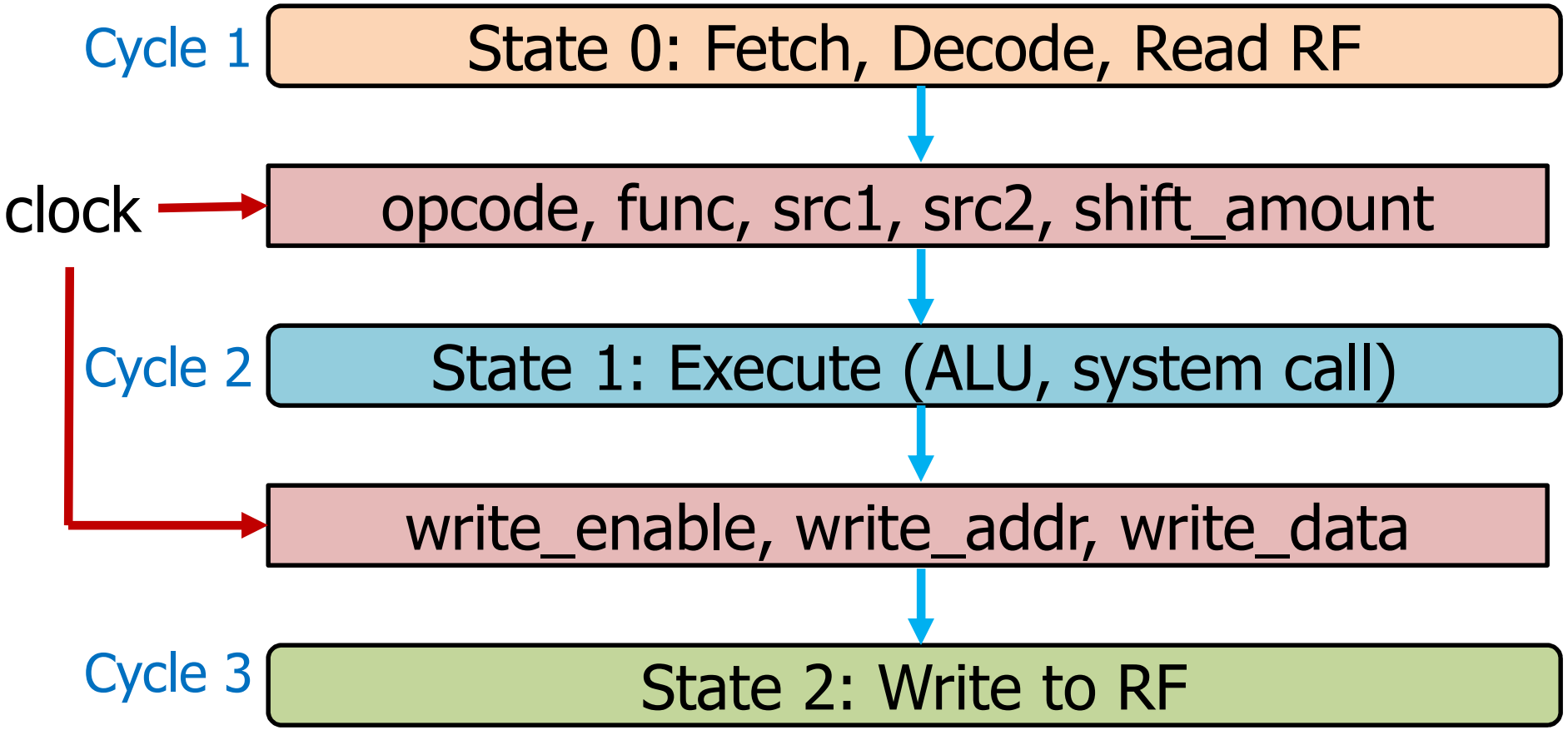
- Two-cycle implementation of a MIPS processor
  - Create AXI4 IP, synthesize, implement
    - Implementation may report timing failure
    - Fails to complete fetch, decode, register read and execution within a cycle
    - Examine the failing paths from implementation timing report
      - Try to understand the longest paths (these are the critical paths of the processor)
    - Move on to the next milestone
  - If you see no timing error, generate bit stream, export hardware, show that your design works by running the Vitis IDE C program

Milestone#3

- Three-cycle implementation of a MIPS processor
  - A natural circuit cut in the two-cycle design is between register read and ALU
    - Allows the processor to use one full cycle to fetch an instruction, decode it, and read register file, and another full cycle to execute the operation in ALU
    - System call execution is also moved to the second cycle
    - This design requires three cycles to complete an instruction
    - A new instruction can be fetched only after the previous instruction is completed
  - Make minimal changes to the modules to implement this idea

Milestone#3

- Three-cycle implementation of a MIPS processor
  - Functional decomposition across cycles in the three-cycle design



Milestone#3

- Three-cycle implementation of a MIPS processor
  - Create AXI4 IP, synthesize, implement, generate bit stream, export hardware, show that your design works by running the Vitis IDE C program
  - If you see timing errors, try to resolve them by optimizing the Verilog code on the critical paths
    - Three-cycle design should comfortably meet the clock cycle time constraint of 10 ns
  - Try different programs to test other instructions that you have implemented
    - For example, replace c=a+b by c=a – b
    - Use logical and shift instructions

Milestone#3

- Three-cycle implementation of a MIPS processor
  - Translate the following program into MIPS ISA and write the corresponding Vitis IDE program to show that it runs correctly on your design

```
int a=-15, b=-40, c, d;
unsigned x=5;
c=a + b;
d=a – b ;
print c;
print d;
c=((a & b) | (a ^ b)) & ~(c | d);
d=((a & 0xabcd) ^ 0xcafe) & (c | 0xdead);
print d;
c=(((a >> x) << 10) << x) ^ ((d >> 10) | (x >> 2));
print c;
```

Milestone#3

- Three-cycle implementation of a MIPS processor
  - A good way to run this program is to do so incrementally
    - You can translate one additional statement or part of it at a time
    - In general, you can print the output of each instruction to check correct progress
    - At the end, you have to demonstrate that the entire program runs correctly producing the expected values for the four print statements
    - The expected results from this program can be found by running it on any computer